MongoDB-Inspired Sharded Document Store

Project 30 — Project Write-up

Team ID: 50

Reddem Bhanu Prakash Reddy 2024101081 | Mule Mohan Venkata Santosh Reddy 2024101067 | Mathamsetty Viswaj 2024101080

1. Project Description

We will build a small distributed JSON document database inspired by MongoDB's sharded-cluster architecture. Each document has a required, sortable `doc_id` field that serves as its *shard key*; documents are partitioned into contiguous `doc_id` ranges, and each range is owned by a **shard** consisting of one **primary** and one **follower**. A stateless **query router** accepts client requests, consults a **persistent shard map** (range → shard, primary, and *ownership epoch*), and forwards them to the right node.

The system supports `insert`, `update`, `delete`, key lookup, and a compact filter-query API (equality, comparison operators {`$eq`, `$gt`, `$lt`, `$in`}, and `$and`/`$or`). Queries that contain the shard key are *routed directly* to one shard; all others use *scatter-gather*, with the router merging, de-duplicating, sorting, and limiting partial results. Writes are replicated primary → follower through an ordered operation log. A coordinator detects primary failure and promotes the follower by incrementing the ownership epoch, which *fences* the old primary against stale writes. Reads are explicitly labelled as either *primary reads* (strongly consistent) or *follower reads* (possibly stale, tagged with replication lag). The demo crashes a primary and migrates one key range between shards while the client workload continues; writes to the migrating range may pause briefly during cutover.

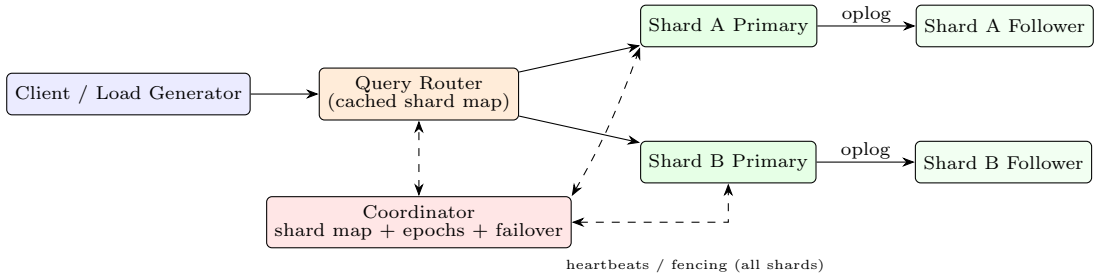


Figure 1: High-level architecture (solid = data path, dashed = control path). The figure shows two shards for simplicity; the actual prototype will use three.

2. Goals

1. **Sharding & storage:** range-partition JSON documents by shard key, with per-shard durable storage (write-ahead log + snapshot) that survives restarts.
2. **Query routing:** direct routing for shard-key queries; scatter-gather with result merging for filter queries on non-key fields.
3. **Replication:** ordered, sequence-numbered operation log from primary to follower, with acknowledgement tracking and measurable replication lag.
4. **Consistency model:** a write is acknowledged only after it is durably appended (`fsync`) to the WAL of *both* primary and follower, so no acknowledged write can be lost; the follower applies logged operations to its in-memory state *asynchronously*, so follower reads can lag (eventually consistent, tagged as potentially stale), while primary reads see all acknowledged writes (per-document linearizable while the primary holds a valid lease). Trade-off: if the follower is unavailable, writes wait or are rejected instead of being acknowledged.
5. **Failover:** heartbeat-based failure detection, follower promotion, durable shard-map update with an incremented epoch, and router refresh; because writes are acknowledged by both replicas, no acknowledged write is lost on a single primary failure. The prototype targets single-node (primary) failure only.
6. **Live migration:** move one key range between shards using snapshot copy → oplog catch-up → short write pause on that range → epoch-bumped cutover → cleanup, while the rest of the client workload continues.
7. **Evaluation:** report query latency, number of documents moved, and recovery time.

3. Tools and Technologies

- **Language:** Python 3.11 with `asyncio` for all concurrent servers.
- **Networking/RPC:** HTTP/JSON over TCP using `FastAPI` (served by `uvicorn`) and `httpx` for inter-node RPC.
- **Persistence:** append-only write-ahead log (JSON lines, `fsync`) plus periodic snapshots; shard map stored as a versioned JSON file updated by atomic rename.
- **Ownership and failover:** a coordinator maintains durable ownership epochs and time-limited leases. Shards reject requests carrying stale ownership information and stop serving writes when their lease expires,

so a recovered old primary cannot accept writes (explicit fencing). Writes are acknowledged only after being durably recorded on both primary and follower. The coordinator is a single process (not itself fault-tolerant, no full consensus); the focus is primary-failure recovery.

- **Indexing & querying:** in-memory ordered key index (`sortedcontainers`) and a custom filter evaluator over JSON documents.
- **Testing & benchmarking:** `pytest`, a custom load generator, fault injection (`SIGKILL`, network partition simulation), `numpy`/`matplotlib` for latency percentiles and plots.
- **Deployment & tooling:** Docker Compose on a single host, Git/GitHub, LaTeX for the report.

4. Special System Requirements

No GPU or specialised hardware is needed. The cluster runs as roughly 8–10 lightweight processes/containers: 1 coordinator, 1–2 routers, and 3 shards × 2 nodes. A single machine with a multi-core CPU, 8 GB RAM, and an SSD (for realistic `fsync` latency) is sufficient. Docker Compose provides isolated nodes and easy fault injection, and the project is reproducible with a single script (`make demo`). If a remote compute environment (RCE) or multi-VM cluster is available, nodes can be spread across hosts to measure realistic network latency; otherwise latency is emulated locally (e.g. `tc netem` or injected delays).

5. Expected Outcomes

Deliverables:

- Working source code for the shard nodes, router, coordinator, and client CLI/API, with setup instructions.
- A persistent shard map with epoch history and durable per-node storage.
- A scripted **demo**: (i) load data across shards; (ii) `SIGKILL` a primary during a continuous write workload and observe automatic failover; (iii) migrate one key range to another shard while writes continue; (iv) show that the fenced old primary rejects writes and that follower reads can lag behind primary reads.
- A test suite covering CRUD correctness, routing, merge semantics, replication, fencing, and migration.
- A final report with architecture, consistency analysis, and evaluation results.

Metrics to be reported (measured over a fixed workload, e.g. 100k documents, mixed 70/30 read/write):

Metric	What is measured	Comparison
Query latency	p50/p95/p99 for key lookup, shard-key query, scatter-gather	Direct routing vs. scatter-gather; primary vs. follower reads
Documents moved	Documents copied, catch-up ops replayed, write-pause duration	Effect of migrated range size
Recovery time	Crash → detection → promotion → first successful write	Effect of heartbeat timeout
Replication lag	Operations or time behind the primary	Replication delay under load

Expected findings: direct routing is markedly faster than scatter-gather (which grows with shard count); follower reads reduce primary load at the cost of bounded staleness; failover completes within a few heartbeat intervals (target of a few seconds) without losing acknowledged writes; and migration completes with only a brief write pause, preserving data integrity (document counts and checksums match before and after).